

Vigilancia EPI

Documentacao de requisitos

Instituicao: SENAI Lauro de Freitas - Bahia

Curso: Desenvolvimento de Sistemas

Entrega: Prototipo funcional para TCC

Projeto: Sistema de vigilancia inteligente para uso de EPI

Data: 07 de junho de 2026

Este documento apresenta o escopo, as regras de negocio, os requisitos funcionais e nao funcionais, os criterios de aceite e as limitacoes do prototipo Vigilancia EPI.

Documentacao de requisitos - Vigilancia EPI

1. Identificacao do projeto

Nome do sistema: Vigilancia EPI

Instituicao: SENAI Lauro de Freitas - BA

Curso: Desenvolvimento de Sistemas

Tipo de entrega: prototipo funcional para TCC

O Vigilancia EPI e um sistema web criado para apoiar a seguranca do trabalho por meio de cameras e inteligencia artificial. A proposta e identificar, em tempo real, se uma pessoa esta usando os EPIs exigidos em uma area monitorada, registrar o status da leitura e disponibilizar essas informacoes para consulta no painel e por API.

O sistema nao substitui o tecnico de seguranca nem a responsabilidade da empresa. Ele funciona como uma ferramenta de apoio, ajudando a perceber situacoes de risco mais rapido do que a fiscalizacao manual conseguiria em todos os pontos ao mesmo tempo.

2. Objetivo dos requisitos

Este documento organiza o que o prototipo precisa fazer, quais limites ele possui e quais condicoes devem ser atendidas para a demonstracao. Os requisitos foram levantados a partir das telas, do backend, da prova de conceito e da arquitetura prevista para evolucao do projeto.

3. Escopo do sistema

3.1 Dentro do escopo

- Cadastro e autenticacao de usuarios.
- Cadastro de EPIs usados pela empresa ou ambiente de teste.
- Upload de fotos de EPIs para treinamento.
- Importacao de dataset anotado no formato YOLO.
- Treinamento de modelos YOLO personalizados por EPI.
- Upload de modelos .pt ja treinados.
- Configuracao de quais itens sao obrigatorios em cada modelo.
- Cadastro de funcionarios com nome, cargo, matricula e fotos.
- Treinamento de reconhecimento de funcionarios como prova de conceito.
- Monitoramento por webcam, camera do navegador, camera remota, RTSP ou video.
- Deteccao de EPIs e classificacao do ambiente como seguro ou perigo.
- Exibicao de FPS, modelo ativo, EPIs exigidos, EPIs ausentes e deteccoes.
- Consulta dos resultados por API externa.
- Registro de backends em um hub central.
- Escolha automatica do backend mais disponivel pelo frontend.
- Conversao para TensorRT quando houver GPU NVIDIA/CUDA e dependencias instaladas.

3.2 Fora do escopo do prototipo

- Substituir procedimentos oficiais de segurança do trabalho.
- Gerar laudos legais de acidente ou documentos jurídicos.
- Garantir reconhecimento facial em nível de produção.
- Fazer controle de ponto, folha de pagamento ou gestão completa de RH.
- Armazenar vídeos longos em nuvem.
- Garantir precisão alta sem dataset adequado e validação em ambiente real.
- Fazer controle avançado de permissões por perfil, como técnico, gestor e administrador.

4. Perfis envolvidos

Perfil	Papel no sistema
Usuario da empresa	Acessa o painel, cadastra EPIs, funcionarios, modelos e cameras.
Tecnico de segurança	Usa os alertas para acompanhar areas de risco e tomar providencias.
Funcionario monitorado	Aparece nas imagens analisadas pelo sistema.
Dispositivo remoto	Envia frames de camera para o backend usando token proprio.
Sistema externo	Consulta a API para obter status de streams, EPIs, funcionarios e deteccoes.
Backend worker	Processa as imagens, executa os modelos e envia metricas ao hub.
Hub central	Mantem a lista de backends disponiveis para o frontend escolher o melhor.

5. Regras de negocio

Codigo	Regra
RN01	O usuario precisa estar autenticado para acessar dados, modelos, EPIs, funcionarios e streams da sua conta.
RN02	Os dados de cada usuario devem ficar separados por conta, evitando que um usuario visualize fotos, modelos ou streams de outro.
RN03	Um EPI precisa ter pelo menos nome para ser cadastrado. Descricao e cor sao informacoes complementares.
RN04	O treinamento simples de EPI so deve iniciar quando houver pelo menos 5 fotos validas.
RN05	Dataset anotado importado deve conter imagens e, quando possivel, labels no formato YOLO.
RN06	Quando nao houver anotacao real da imagem, o prototipo pode usar uma marcacao simplificada para demonstrar o fluxo de treinamento.
RN07	Um funcionario precisa ter nome para ser cadastrado. Cargo e matricula sao opcionais no prototipo.
RN08	O treinamento de reconhecimento de funcionarios exige pelo menos 3 fotos de rosto por funcionario participante.
RN09	Modelos globais podem ser usados como base, mas modelos enviados ou treinados pelo usuario ficam vinculados ao proprio usuario.
RN10	Cada modelo pode ter uma lista propria de EPIs obrigatorios. Essa lista define o que sera considerado ausente durante a leitura.
RN11	O status deve ser considerado "perigo" quando uma pessoa for detectada e algum item obrigatorio estiver ausente.
RN12	O status deve ser considerado "seguro" quando nao houver item obrigatorio ausente na leitura atual.
RN13	Cameras remotas usam token especifico e nao podem executar acoes administrativas da conta.
RN14	A API externa so pode retornar dados da conta autenticada.
RN15	A conversao TensorRT so deve ser oferecida quando o backend possuir NVIDIA/CUDA e as dependencias necessarias.
RN16	Se o modelo TensorRT falhar ao carregar, o backend deve voltar para o modelo .pt original.
RN17	O hub deve ignorar backends que deixaram de enviar heartbeat dentro do tempo configurado.

6. Requisitos funcionais

6.1 Acesso e usuarios

Codigo	Requisito	Prioridade	Situacao no prototipo
RF01	Permitir cadastro de usuario com nome, documento, telefone, email, setor e senha.	Alta	Implementado
RF02	Permitir login usando credencial e senha, retornando token de sessao.	Alta	Implementado
RF03	Permitir logout do usuario autenticado.	Media	Implementado
RF04	Permitir consultar os dados do usuario logado.	Media	Implementado
RF05	Permitir bloqueio e restricoes de usuarios em rotas administrativas.	Baixa	Parcial

6.2 EPIs

Codigo	Requisito	Prioridade	Situacao no prototipo
RF06	Permitir cadastrar EPIs com nome, descricao e cor.	Alta	Implementado
RF07	Permitir listar, editar e remover EPIs cadastrados.	Alta	Implementado
RF08	Permitir upload de varias fotos para um EPI.	Alta	Implementado
RF09	Permitir remover fotos de um EPI.	Media	Implementado
RF10	Permitir importar dataset anotado em .zip para melhorar o treinamento.	Media	Implementado
RF11	Permitir treinar um modelo YOLO personalizado para um EPI.	Alta	Implementado como POC
RF12	Atualizar o status do EPI quando um novo modelo for treinado.	Media	Implementado

6.3 Funcionarios

Codigo	Requisito	Prioridade	Situacao no prototipo
RF13	Permitir cadastrar funcionarios com nome, cargo e matricula.	Alta	Implementado
RF14	Permitir listar, editar e remover funcionarios.	Alta	Implementado
RF15	Permitir upload de fotos de rosto para cada funcionario.	Alta	Implementado
RF16	Permitir remover fotos de funcionarios.	Media	Implementado
RF17	Permitir treinar um modelo para reconhecer funcionarios cadastrados.	Media	Implementado como POC
RF18	Exibir o funcionario identificado quando o modelo de reconhecimento estiver disponivel.	Media	Implementado

6.4 Modelos de IA

Codigo	Requisito	Prioridade	Situacao no prototipo
RF19	Listar modelos globais e modelos do usuario logado.	Alta	Implementado
RF20	Permitir upload de modelo .pt pelo usuario.	Alta	Implementado
RF21	Permitir remover modelos enviados pelo proprio usuario.	Media	Implementado
RF22	Permitir definir um modelo como padrao da conta.	Media	Implementado
RF23	Permitir configurar EPIs obrigatorios por modelo.	Alta	Implementado
RF24	Ler as classes existentes no modelo para auxiliar a configuracao.	Media	Implementado
RF25	Permitir consultar o status de conversao TensorRT de um modelo.	Media	Implementado
RF26	Permitir iniciar conversao TensorRT manualmente.	Media	Implementado
RF27	Iniciar conversao TensorRT automaticamente apos upload ou treinamento quando o backend permitir.	Baixa	Implementado

6.5 Cameras e streams

Codigo	Requisito	Prioridade	Situacao no prototipo
RF28	Permitir adicionar streams de camera no painel.	Alta	Implementado
RF29	Permitir remover streams ativos.	Alta	Implementado
RF30	Permitir iniciar camera pelo navegador.	Alta	Implementado
RF31	Permitir camera remota por pagina separada (/cam).	Alta	Implementado
RF32	Permitir transmissao de frames por duas rotas paralelas para reduzir atraso.	Media	Implementado
RF33	Permitir uso de camera RTSP ou URL HTTP/HTTPS.	Media	Implementado
RF34	Permitir upload de video para demonstracao ou teste.	Media	Implementado
RF35	Permitir localizar cameras de rede com varredura ONVIF/RTSP.	Baixa	Implementado
RF36	Permitir informar credenciais de camera quando necessario.	Baixa	Implementado
RF37	Permitir pausar ou parar o envio de camera externa.	Media	Implementado

6.6 Processamento e alertas

Codigo	Requisito	Prioridade	Situacao no prototipo
RF38	Processar frames em tempo real usando YOLO.	Alta	Implementado
RF39	Escolher automaticamente CPU, DirectML ou CUDA conforme disponibilidade.	Alta	Implementado
RF40	Usar TensorRT como runtime quando houver .engine compativel.	Media	Implementado
RF41	Gerar imagem anotada com as deteccoes do modelo.	Alta	Implementado
RF42	Exibir status "seguro" ou "perigo" no painel.	Alta	Implementado
RF43	Exibir quais EPIs obrigatorios estao ausentes.	Alta	Implementado
RF44	Exibir FPS do processamento.	Media	Implementado
RF45	Exibir classes detectadas, confianca e caixa de deteccao.	Media	Implementado na API/status
RF46	Exibir atividade interpretada, como funcionario identificado em situacao de risco.	Media	Implementado

6.7 API de integracao

Codigo	Requisito	Prioridade	Situacao no prototipo
RF47	Disponibilizar rota para listar streams da conta autenticada.	Alta	Implementado
RF48	Disponibilizar rota para consultar deteccoes de um stream especifico.	Alta	Implementado
RF49	Disponibilizar rota para listar funcionarios cadastrados.	Media	Implementado
RF50	Disponibilizar rota para listar EPIs cadastrados.	Media	Implementado
RF51	Retornar respostas em JSON para integracao com sistemas externos.	Alta	Implementado
RF52	Impedir que a API consulte stream fora da conta autenticada.	Alta	Implementado

6.8 Hub e multiplos backends

Codigo	Requisito	Prioridade	Situacao no prototipo
RF53	Permitir que um backend worker registre URL publica, hardware e disponibilidade no hub.	Media	Implementado
RF54	Permitir heartbeat periodico do backend para manter o registro ativo.	Media	Implementado
RF55	Listar backends online em ordem de melhor disponibilidade.	Media	Implementado
RF56	Permitir que o frontend escolha automaticamente um backend pelo hub.	Media	Implementado

Codigo	Requisito	Prioridade	Situacao no prototipo
RF57	Expor status de hardware, metricas e disponibilidade do backend.	Media	Implementado

7. Requisitos nao funcionais

Codigo	Requisito	Prioridade
RNF01	A interface deve funcionar em navegador moderno, sem exigir instalacao no computador do usuario final.	Alta
RNF02	O sistema deve ter interface responsiva para uso em notebook, desktop e celular.	Media
RNF03	O processamento deve buscar baixa latencia para permitir percepcao quase imediata do alerta.	Alta
RNF04	O backend deve conseguir rodar localmente, pois o processamento de video pode exigir hardware proprio.	Alta
RNF05	O backend deve suportar CPU, GPU via DirectML e GPU NVIDIA/CUDA, conforme disponibilidade.	Alta
RNF06	Em ambiente NVIDIA, o sistema deve tentar usar TensorRT para melhorar desempenho quando configurado.	Media
RNF07	As APIs devem usar JSON e aceitar frontend separado por CORS.	Alta
RNF08	O sistema deve manter separacao de dados por usuario.	Alta
RNF09	Tokens de sessao e tokens de camera devem ser exigidos nas rotas protegidas.	Alta
RNF10	O codigo deve permanecer modular, separando frontend, backend, processamento, usuarios, streams e hub.	Media
RNF11	O sistema deve permitir execucao em ambiente gratuito ou de baixo custo para apresentacao do TCC.	Media
RNF12	O hub deve desconsiderar backends inativos para evitar que o frontend escolha servidor indisponivel.	Media
RNF13	O painel deve mostrar informacoes suficientes para o usuario entender o estado da leitura sem abrir logs tecnicos.	Alta
RNF14	O sistema deve aceitar evolucao para varias cameras e varios backends sem mudar a ideia principal da arquitetura.	Media
RNF15	Fotos de funcionarios e imagens de ambiente real devem ser usadas apenas com autorizacao.	Alta

8. Requisitos de hardware e software

8.1 Ambiente minimo para demonstracao

- Notebook ou PC com processador moderno.
- 8 GB de RAM.
- Webcam, camera do celular ou video de teste.
- Navegador atualizado.
- Python com dependencias do projeto instaladas.
- Modelos YOLO disponiveis na pasta models/.

8.2 Ambiente recomendado

- Processador Intel i5/i7 ou AMD Ryzen 5/7.
- 16 GB de RAM.
- SSD.

- Camera Full HD, camera IP ou celular com boa iluminacao.
- GPU dedicada quando disponivel.
- Rede local estavel para cameras remotas.

8.3 Ambiente de alto desempenho

- GPU NVIDIA RTX, como RTX 2060 ou superior.
- CUDA configurado.
- TensorRT e ONNX instalados quando a demonstracao incluir otimizacao.
- Backend dedicado para inferencia.
- Uma ou mais cameras IP/RTSP.

8.4 Hospedagem prevista

Parte	Ambiente sugerido
Frontend	Vercel ou servidor estatico equivalente.
Backend worker	Maquina local ou servidor com Python e acesso a camera/modelos.
Hub	Render ou outro servico leve para Flask.
Exposicao publica do worker	Cloudflare Tunnel para teste e demonstracao.

9. Requisitos de dados

- Imagens dos EPIs em angulos, iluminacoes e distancias diferentes.
- Fotos dos funcionarios com autorizacao de uso.
- Datasets anotados quando for necessario melhorar a precisao.
- Classes dos modelos alinhadas com os nomes usados nas regras de EPI obrigatorio.
- Politica de cuidado com imagem, principalmente em ambiente real de trabalho.
- Dados separados por usuario em dados/users/<uid>/.

10. Casos de uso principais

UC01 - Cadastrar EPI

1. Usuario acessa o painel autenticado.
2. Abre a aba de EPIs.
3. Informa nome, descricao e cor.
4. Salva o cadastro.
5. O sistema exibe o novo EPI na lista.

UC02 - Treinar modelo de EPI

1. Usuario cadastra um EPI.
2. Envia pelo menos 5 fotos ou importa um dataset anotado.
3. Clica em treinar modelo.
4. O backend inicia o treinamento em segundo plano.

5. Ao final, o modelo aparece como treinado e pode ser usado em uma camera.

UC03 - Monitorar camera

1. Usuario abre a aba de cameras.
2. Adiciona uma camera do navegador, remota, RTSP ou video.
3. Escolhe o modelo e os EPIs obrigatorios.
4. O backend processa os frames.
5. O painel mostra imagem anotada, status, FPS e itens ausentes.

UC04 - Demonstrar alerta de risco

1. Usuario inicia a camera.
2. Mostra a pessoa usando o EPI obrigatorio.
3. O painel deve ficar como seguro.
4. Usuario remove o EPI ou troca para uma cena sem o item.
5. O painel deve indicar perigo e listar o EPI ausente.

UC05 - Consultar deteccoes por API

1. Sistema externo envia requisicao autenticada.
2. Backend valida o token.
3. Sistema informa o stream desejado.
4. Backend retorna status, deteccoes, funcionarios, EPIs ausentes, FPS e runtime.

UC06 - Usar hub para escolher backend

1. Backend worker inicia e registra sua URL publica no hub.
2. Worker envia heartbeat com metricas.
3. Frontend consulta o hub.
4. Hub retorna os backends online ordenados.
5. Frontend usa o melhor backend disponivel.

11. Criterios de aceite para a banca

Item avaliado	Critério de aceite
Login	O usuario consegue entrar no sistema e visualizar o painel.
EPIs	O usuario consegue cadastrar EPI e enviar fotos.
Modelos	O usuario consegue listar modelos e selecionar um modelo para a camera.
Camera	A imagem da camera ou video aparece no painel.
Deteccao	O sistema mostra deteccoes do modelo na imagem processada.
Alerta	O painel indica "perigo" quando item obrigatorio esta ausente.
Status seguro	O painel volta para "seguro" quando nao ha item obrigatorio ausente.
FPS	O painel informa taxa de processamento.

Item avaliado	Critério de aceite
Funcionarios	O sistema permite cadastro e fotos de funcionario; reconhecimento pode ser demonstrado se o modelo estiver treinado.
API	Uma consulta retorna JSON com stream, status, detecoes e EPIs ausentes.
Hub	O backend consegue aparecer como disponivel no hub quando configurado.
TensorRT	Em maquina NVIDIA, a tela/API mostra possibilidade de conversao ou runtime otimizado.

12. Limitacoes conhecidas do prototipo

- A precisao depende diretamente da qualidade do modelo e das imagens usadas no treinamento.
- Poucas fotos, baixa iluminacao e angulos ruins reduzem a confiabilidade.
- O reconhecimento de funcionarios foi implementado como POC com YOLO; para producao seria melhor usar uma solucao propria de reconhecimento facial com politicas claras de privacidade.
- O uso de Cloudflare Tunnel em teste pode mudar a URL publica.
- Hospedagens gratuitas podem entrar em modo de espera ou ter limite de desempenho.
- O treinamento local pode demorar, principalmente em CPU.
- A decisao "seguro" ou "perigo" e uma leitura automatica de apoio, nao uma conclusao legal sobre a area.

13. Rastreabilidade com partes do sistema

Parte do sistema	Requisitos relacionados
frontend/index.html	RF01 a RF04, RF06 a RF27, RF28 a RF46, RF56
frontend/cam.html	RF31, RF32, RF37
frontend/api/config.js	RF56
backend/app.py	RF01 a RF57
backend/processor.py	RF38 a RF46
backend/multistream.py	RF32
backend/users.py	RF01 a RF05, RN01, RN02
backend/camera_discovery.py	RF35, RF36
backend/hub/hub.py	RF53 a RF56
dados/users/	RN02, requisitos de dados

14. Melhorias futuras

- Criar perfis de acesso separados, como administrador, tecnico e visualizador.
- Adicionar historico persistente de alertas por data, camera e funcionario.
- Criar dashboard com indicadores de recorrencia de falta de EPI.
- Melhorar reconhecimento de funcionarios com tecnica dedicada de embeddings faciais.
- Criptografar tokens sensiveis e implementar rotacao de chaves.
- Criar logs auditaveis para uso em empresas.
- Permitir configuracao de areas de interesse dentro da imagem.
- Integrar notificacoes por email, WhatsApp ou sistema interno.
- Criar instalador ou container Docker para facilitar deploy.